Identifiability and Confounding in Test-Adequacy Metric Validity Studies: A Cross-Language Empirical Analysis

Junjie Chen[a]

[a]*School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, China*

Abstract

A large body of software engineering research asks whether a test-adequacy metric predicts a suite's ability to detect real faults. The standard argument regresses fault detection on existing metrics, adds the candidate metric, and reads the cross-validated R^2 gain as incremental value. We ask whether that evidence can be resolved at the sample sizes the literature uses. We build test-suite subsets on Defects4J (Java) and BugsInPy (Python), seven projects each, derive each subset's fault-detection rate from per-fault trigger-test metadata without executing the suites, and quantify the incremental R^2's uncertainty under several evaluation schemes. We find: (i) the incremental R^2 of a behavioural-matching feature block has a 95% interval width of 0.056 on Defects4J and 0.143 on BugsInPy under version-level resampling, and of 0.120 and 0.146 under a project-level cluster bootstrap, with an unstable sign on Defects4J; (ii) a simulation calibrated on the observed data shows that a feature with zero within-project effect whose project means correlate with baseline detection rates yields a non-trivial increment in up to 82% of unadjusted pooled analyses, against below 1% under project fixed effects; (iii) behavioural features identify the source project with 0.895 and 0.824 accuracy against a uniform-random baseline of 1/7; and (iv) resolving a 0.05 target half-width needs on the order of 100 to 150 independent fault versions and 0.02 on the order of 200 to 300, depending on the benchmark and the resampling hierarchy. We conclude that incremental-validity claims at the magnitudes common in this literature are not identifiable at conventional

Email address: junjie_chen@njust.edu.cn (Junjie Chen)

sample sizes, and provide a protocol and resolution curves for sizing future studies.

Keywords: test adequacy, software metrics, empirical software engineering, confounding, cross-project evaluation

---

1. Introduction

Software engineering research has produced a large family of test-adequacy metrics: statement and branch coverage, mutation score, checked coverage, oracle gap, pseudo-tested methods, semantic coverage, and behavioural verification gaps, among others. For each of them the central validity question is the same: *does the metric carry information about whether a test suite will detect real faults, beyond what simpler and cheaper metrics already provide?*

The prevailing way to answer this question is incremental-prediction analysis. A fault-detection outcome is regressed on a baseline set of metrics, the candidate metric is added, and the resulting gain in explained variance—usually reported as an increment in R^2, or as the coefficient and p-value of the added predictor—is interpreted as evidence of incremental value.

This paper asks a prior question: *is that evidence statistically resolvable at the sample sizes the literature uses?* We use *resolvable* (and, interchangeably, *identifiable at the design level*) in an operational rather than a causal-inference sense: an incremental effect is resolvable when the uncertainty of the estimated increment, together with its sensitivity to defensible analysis choices, is small relative to the effect magnitude the study claims to have established. Two properties of the setting make the answer non-obvious.

First, the units of analysis are not independent. A study that draws many test suites from a small number of subject programs, or many subsets from each of a few fault versions, accumulates nominal sample size without accumulating independent information. Significance tests computed over nominal instances overstate precision, and cluster resampling can change the intervals substantially—in our data, by a factor of about two on Defects4J.

Second, project identity is a strong common cause. Style, size, coverage instrumentation, and fault composition all vary systematically across subject programs, and so do the distributions of almost any derived metric. A metric that merely distinguishes projects will appear to predict fault detection in any pooled analysis, even if it has no effect within any project.

Neither concern is exotic, and both are discussed in the statistical literature. What is missing is a quantitative account of how much they matter for the specific claims this community makes. This paper supplies that account, and it does so on two benchmarks in two languages with a common outcome construction.

Contributions.

1. **A resolution analysis of incremental-validity claims** (Section 4). On two benchmarks we show that the incremental R^2 of a behavioural-matching feature block is of the same order as its own uncertainty, that its sign on Defects4J depends on the cross-validation scheme, and—using resolution curves—how many independent fault versions a study needs in order to resolve effects of 0.10, 0.05, and 0.02.
2. **A mechanism and its rate** (Section 5). A simulation calibrated on the observed data isolates project-level confounding as a driver, and measures the rate at which the unadjusted analysis reports an increment as a function of confounding strength ρ and sample size. Because ρ is directly estimable from project-level means, the sensitivity analysis is cheap.
3. **A cross-language replication** (Section 6). The project-fingerprint phenomenon appears on both Defects4J (Java) and BugsInPy (Python), with identical random baselines, and the pooled analysis favours the larger model on both.
4. **A robustness analysis over analytic choices** (Section 7). Leave-one-project-out, removal of the dominant project, project-size thresholds, and two bootstrap hierarchies, each reported with the effect on the conclusion.
5. **A pre-submission protocol** (Section 8) with five checks, none of which requires executing the test suites under study.

2. Background and Related Work

2.1. Test-adequacy metrics and their validity

Coverage-based metrics are the oldest and most widely used. Inozemtseva and Holmes [1] studied the correlation between coverage and fault detection over 31,000 test suites and five real faults, and found the correlation to be weak and strongly conditioned on suite size; the result has been a touchstone

for the claim that coverage is an insufficient measure of test quality. Mutation score was argued to be a better predictor by Papadakis et al. [2], who analysed correlation with real fault detection under controlled conditions and carefully separated masking effects from genuine detection.

A second family measures what the tests actually *check*. Checked coverage [3] measures the fraction of covered statements that influence an assertion through the dynamic slice. Pseudo-tested methods [4] identify methods whose bodies can be deleted without any test failing; Vera-Pérez et al. [5] replicated the phenomenon at scale. The "oracle gap" [6] contrasts coverage with mutation score to locate oracle weaknesses. Semantic coverage measures failure-revealing capability directly: it was introduced as a semantics-based measure of test-suite effectiveness by Al Blwi et al. [7] and developed further by Ayad et al. [8]. Paul and Holmes [9] extract expected behaviours from documentation and source and measure how many are never verified by any test.

These works are the immediate context for ours. Two shared features matter here: their validity arguments are frequently explanatory rather than predictive—the metric is argued to capture something meaningful, rather than being shown to predict fault detection at a stated confidence level—and they are rarely compared with each other under a common outcome variable. Our contribution is orthogonal: we do not propose a new metric, but examine whether the inferential machinery used to validate metrics can support the conclusions drawn from it.

2.2. Statistical practice in multi-project empirical studies

The concerns we quantify are instances of long-standing issues in applied statistics. Non-independence of observations drawn from clusters motivates cluster-robust standard errors and mixed-effects models [10]. Mixed-effects models are established tools used in empirical software engineering to account for repeated or nested observations [11], but they are applied unevenly. Confounding by group identity is the setting of Simpson's paradox [12], and the fixed-effects estimator that removes it is textbook econometrics [13]. Publication and reporting practice—in particular the tendency to report point estimates and p-values without any statement of the precision the design achieved—is the subject of a broad meta-scientific literature [14]; the specific recommendation we arrive at, that a study state the smallest effect its design could have detected, is the standard power-analysis requirement [15] applied in the mode of sensitivity analysis [16].

Our contribution is not the introduction of these tools but their calibration for this community's claims: we measure the magnitudes, on real benchmarks, and show that at conventional sample sizes the corrected comparison cannot resolve the effects being claimed. We also show that the choice of resampling unit—individual fault versions or whole projects—changes the reported uncertainty substantially, which is a practical matter rarely addressed in this literature.

Hierarchical structure, clustered inference, and effective sample size. Observations in this setting are nested—subsets within fault versions within projects—and the nesting is not incidental: it determines how much independent information a study actually contains. Cluster-robust inference [10] and effective-sample-size adjustments are the standard responses, and grouped ("leave-group-out") cross-validation is the analogous correction at the model-selection stage: randomly splitting nested observations leaks group-level information into the test folds and yields optimistically biased estimates of out-of-sample performance [17]. We use grouped cross-validation throughout for this reason, and report both instance-level and version-grouped versions of the project-classification diagnostic to show the difference it makes in our setting.

Fixed effects versus random effects. Group identity can be handled either by fixed effects, which estimate a separate intercept per group and identify the effect purely from within-group variation, or by random effects, which model the group intercepts as draws from a distribution and can in principle recover between-group information [13]. We use fixed effects because the question we ask is explicitly within-project—whether a metric helps discriminate suites of the same project—and because with seven groups the random-effects variance components would themselves be poorly determined. Identification of a within-project effect requires a project to contribute at least two fault versions: a project contributing one version supplies a level, not a contrast. On Defects4J every project contributes at least five versions and the condition holds throughout. On BugsInPy exactly one project fails it, and we quantify the consequences in Section 7 rather than leaving them implicit.

The bootstrap with few clusters. Cluster resampling is unreliable when the number of clusters is small, and the resulting intervals can be too narrow rather than too wide [18]. With seven projects per benchmark our project-level intervals fall squarely in that regime, which is why we present them as

a sensitivity analysis rather than as calibrated confidence intervals, and why we report the version-level hierarchy alongside them rather than choosing one.

Project heterogeneity in empirical software engineering. The statistical points above are not new to software engineering practice, even if their magnitudes for metric-validity claims are not usually measured. Cross-project and within-project analyses are known to produce different answers: cost-estimation studies show systematic differences between cross-company and within-company models [19], and cross-project defect prediction degrades sharply when the source and target projects differ in domain or process [20], with process and organisational factors explaining as much as code metrics [21]. Our contribution is to quantify the analogous effect for test-adequacy metrics, and to show that in this setting the within-project comparison cannot resolve the effects being claimed.

2.3. Benchmarks

Defects4J [22] provides real Java faults with per-fault trigger tests, which makes a fault-detection outcome computable without executing the suites under study. BugsInPy [23] provides the analogous Python resource: each fault records the single test that triggers it. The availability of trigger metadata in both benchmarks is what makes a common outcome variable possible across languages.

3. Study Design

3.1. Unit of analysis

The unit is a *test-suite subset*: a fraction of a project's test suite at a specific fault version. Varying the fraction gives within-version variation in suite size and in composition, which is what the incremental analysis requires. We treat fault *versions*, not subsets, as the independent resampling units in the version-level analysis, while recognising that they remain clustered within projects; a project-level hierarchy is reported alongside it (Section 7). We report both the nominal sample size and the number of fault-version units throughout, using the latter as a count of units rather than as a formal effective sample size (Section 8).

Table 1: Analysis sample. A *version* is a `(project, fault)` pair with at least one usable subset; an *instance* is one subset run. Counts are those that enter the analysis. Of the 626 subset records collected on Defects4J, 355 carried both metrics and 352 entered the analysis, three more having been dropped because their assertion features failed to parse; on BugsInPy 245 of 246 instances entered.

Benchmark	Project	Versions	Instances	Mean line coverage (%)
Defects4J	Chart	14	49	27.4
	Cli	6	12	49.4
	Codec	5	10	23.3
	Compress	17	33	37.6
	Csv	5	7	50.3
	Lang	38	131	31.2
	Math	54	110	33.2
BugsInPy	black	18	54	83.1
	fastapi	3	9	86.0
	httpie	2	6	77.0
	sanic	1	3	71.3
	tornado	6	17	38.1
	tqdm	9	27	70.3
	youtube-dl	43	129	29.1

The version counts differ from the number of faults available in each project: they are the versions for which a usable subset could be constructed. Coverage of either benchmark is therefore partial. On Defects4J the analysed versions span the seven projects listed; the remaining ten Defects4J projects were not collected. On BugsInPy, the usable versions per project range from 1 (sanic) to 43 (youtube-dl).

3.2. Benchmarks and subjects

We use seven projects from each benchmark (Table 1). On Defects4J we measure on the *fixed* version of each fault, because mutation analysis aborts on versions whose trigger tests fail; on BugsInPy we likewise use the fixed version so that the suite pool is well defined. All analyses are therefore about suite variation at a stable program revision, not about the fault itself.

3.3. Constructing subsets

Defects4J's coverage and mutation commands do not accept an explicit test list, and the framework rewrites `all_tests`. We therefore exploit the fact that each project's build files define the test sources with a fileset over

the *test source directory*: moving the sources of non-selected tests out of that directory causes the framework to build and run only the selected subset. Two details were necessary in practice: (i) test classes reference each other, so the selected set must be closed under a dependency closure; (ii) aggregating test classes (e.g. `*PackageTests`) are compiled even when not run, so failure to restore them breaks the build. We restore missing sources adaptively by parsing `javac`'s missing-symbol errors and copying the named classes back from a staging area, iterating until compilation succeeds.

On BugsInPy, subsets are far simpler: `pytest` accepts an explicit list of test node identifiers, so no source relocation is needed. The pool for a version is the set of collected tests that pass at the fixed revision.

Sampling is deterministic and fixed in advance. For each version we draw subsets at fractions of the runnable pool—0.10 and 0.25 for the main Defects4J collection, plus a remainder of pilot instances at 0.05, 0.50 and 1.0, and 0.10, 0.30 and 0.60 on BugsInPy—with target size $\max(1, \mathrm{round}(n \cdot f))$ and uniform sampling *without* replacement from the pool, seeded from the string `project|bug|f|rep` so that any single instance can be regenerated in isolation. Subsets at different fractions of the same version are drawn independently and are *not* nested. The Defects4J pool is the framework's test list at the fixed revision, extended by the dependency closure described above; the BugsInPy pool is the set of tests that pass at that revision, built once per project from an explicit file list, because some projects' test modules do not match `pytest`'s default collection patterns.

Mutation score is measured on 15 classes per version. The classes are drawn once per project with a fixed seed and reused wherever they are present, so that the score is comparable across a project's versions; where a version has been repackaged and the cached classes are absent (108 of the 355 analysed instances), the sample is redrawn from that version's own classes and the record notes which source was used.

3.4. Outcome variable: differential fault detection

For a subset S of project P's test suite, the outcome is

$$\mathrm{detect}(S) \;=\; \frac{\left|\{\, f \in \mathcal{F}_P \;:\; \mathrm{trigger}(f) \in S \,\}\right|}{\left|\mathcal{F}_P\right|}, \tag{1}$$

where $\mathcal{F}_P$ is the set of faults of P and $\mathrm{trigger}(f)$ is the test recorded as triggering fault f. The outcome is computed purely from metadata: *no suite under study is executed to obtain it.*

Two features of this definition matter and we state them explicitly. First, the denominator is the project's *whole* fault set, not the faults fixed at the revision the subset was drawn from: on Defects4J only about half of a project's faults have their trigger class present in any one revision's test list. Second, matching is at class granularity, so a fault counts as detected when the subset contains its trigger class, whether or not that revision also carries the specific trigger method. We adopt this project-level denominator deliberately—it asks how well a suite discriminates among the project's known faults and gives the outcome a target that does not depend on which fault the revision happens to fix—but it is a choice, and Section 7 reports what changes under the revision-matched alternative.

This outcome does *not* eliminate the clean-program assumption inherent in trigger-test-based fault detection [24]: the metadata records which test the fault's patch makes fail, not whether a passing suite would have revealed the fault.

Two calibration details apply. On BugsInPy, trigger tests are occasionally declared on a `unittest` base class that `pytest` does not collect directly (only its subclasses); we therefore match triggers at the level of (file, method name) and restrict $\mathcal{F}_P$ to faults whose trigger falls inside the runnable pool—otherwise faults carried by network-dependent tests would depress the outcome systematically.

3.5. Predictors

- **Size**: log of the number of tests actually run.
- **Coverage**: line coverage of the project's own sources (Defects4J: all main classes instrumented; BugsInPy: the project package plus top-level modules).
- **Mutation score**: Defects4J only, measured on a paired sample of 15 classes per version for cost reasons. BugsInPy has no comparable low-cost mutation toolchain, so the classical block there is size + coverage.
- **Behavioural matching** (the candidate block): assertion-level features—assertions per test method, ratio of strong to weak assertions, ratio of predicate-form assertions, and distinct calls per assertion. These are syntactic proxies for the semantic notion of "the test asserts what the code is supposed to do".

3.6. Analysis

We compare four nested models: size; the classical block; size + behaviour; and all predictors. Comparisons use K-fold cross-validation *grouped by fault version*, so that no version contributes to both training and test folds. We report the result with and without project fixed effects, and, as a sensitivity check on the fold construction, under leave-one-version-out cross-validation.

Uncertainty is estimated by bootstrap. Because the choice of resampling unit is consequential and rarely stated in this literature, we report *two* hierarchies throughout: resampling individual fault versions, and resampling whole projects as clusters. The second reflects a more conservative dependence structure, treating projects rather than fault versions as the independent resampling units; with only seven projects per benchmark it is also the more variable estimator, and we interpret it as a sensitivity analysis rather than as a definitive frequentist confidence interval.

Estimator for the cross-validated R^2. Throughout we report the pooled (nested) form

$$R^2_{\mathrm{CV}} \;=\; 1 - \frac{\sum_{i=1}^{n} \big(y_i - \hat{y}_{i,\,-g(i)}\big)^2}{\sum_{i=1}^{n} \big(y_i - \bar{y}\big)^2}, \tag{2}$$

where $g(i)$ is the fault version of observation i, $\hat{y}_{i,-g(i)}$ is its prediction from the model fitted on all folds except the one holding out $g(i)$, and $\bar{y}$ is the grand mean. We do not average per-fold R^2 values, which can be dominated by folds with little outcome variance and are not comparable across resamples.

Bootstrap specification. Each replicate resamples units with replacement—fault versions for the version-level hierarchy, whole projects for the project-level one—and recomputes ΔR^2 on the resampled data using the same grouped K-fold scheme, with folds recomputed within each replicate. Intervals are percentile intervals from $B = 2000$ replicates, so that each tail of the 95% interval is supported by about 50 replicates rather than the handful that $B = 200$ would provide; the resolution curves in Table 4 use $B = 1000$ replicates per point. We also report the fraction of replicates in which $\Delta R^2 \leq 0$. Replicates yielding fewer than six versions are discarded: six is the smallest number for which the grouped five-fold cross-validation used in every comparison is defined. Point estimates and bootstrap means are reported separately: the former is the statistic computed on the full sample, the latter the mean of the resampling distribution, and they need not coincide.

Table 2: Cross-validated R^2. Grouped CV holds out whole fault versions; FE adds project dummies; LOVO is leave-one-version-out with FE. The classical block is size + coverage + mutation (Defects4J) or size + coverage (BugsInPy).

Benchmark	Scheme	Size	Classical	Size+beh.	Full
Defects4J	Pooled	0.128	0.292	0.209	**0.379**
	Grouped CV, FE	0.556	**0.594**	0.553	0.583
	LOVO, FE	0.552	**0.591**	0.557	0.591
BugsInPy	Pooled	0.305	0.297	0.300	**0.315**
	Grouped CV, FE	0.315	0.383	0.314	**0.398**
	LOVO, FE	0.306	0.378	0.328	**0.404**

With only seven project clusters per benchmark, the project-level intervals are themselves imprecisely estimated; the project-level bootstrap is therefore used to expose sensitivity to the assumed independent unit, not to claim a well-calibrated seven-cluster confidence interval.

4. RQ1: To what extent can incremental-validity effects be resolved at realistic sample sizes?

4.1. Model comparison under three evaluation schemes

Table 2 reports cross-validated R^2 for the four models under three schemes.

The pooled analysis—the analysis most commonly reported—selects the full model on *both* benchmarks, by +0.087 and +0.018 respectively. Introducing project fixed effects reverses the sign on Defects4J ($+0.087 \to -0.011$) but leaves the advantage essentially intact on BugsInPy ($+0.018 \to +0.015$). Leave-one-version-out agrees with grouped cross-validation on the ranking but not on the magnitude: under LOVO the Defects4J decrement shrinks to -0.001, an order of magnitude smaller than under grouped CV.

4.2. Uncertainty of the increment

Table 3 gives the increment and its bootstrap interval under both resampling hierarchies.

Three observations. The intervals are wide relative to the effects at stake: even the narrower Defects4J version-level interval, ± 0.028, cannot distinguish "no effect" from a 0.05 increment in either direction. The resampling hierarchy matters: on Defects4J the cluster interval is about twice as wide as

Table 3: ΔR^2 between the full model and the classical block, with project fixed effects. The point estimate and the bootstrap mean are different quantities and are reported separately. The last column, P_{boot}, is the share of bootstrap replicates with $\Delta R^2 \leq 0$ (percentile intervals from $B = 2000$ replicates).

Benchmark	Unit	Point	Boot. mean	95% CI	Width	P_{boot}
Defects4J	Version	-0.011	-0.009	$[-0.038, +0.017]$	0.056	0.78
	Project	-0.011	$+0.004$	$[-0.052, +0.069]$	**0.120**	0.43
BugsInPy	Version	$+0.015$	$+0.019$	$[-0.047, +0.096]$	0.143	0.29
	Project	$+0.015$	$+0.034$	$[-0.021, +0.125]$	0.146	0.10

the version-level interval (0.120 against 0.056), and on BugsInPy the two are nearly equal (0.143 and 0.146). The direction of the change differs from the usual expectation because with only seven project clusters the cluster bootstrap is itself highly variable. And the two benchmarks do not agree on the sign of the point estimate (-0.011 versus $+0.015$), which is consistent with both being indistinguishable from zero rather than with a genuine language difference.

4.3. How much sample is enough? Resolution of the estimate

Rather than extrapolating a single sample size, we report the *resolution* of the estimate—the half-width of the 95% bootstrap interval—as a function of the number of independent fault versions (Table 4). We deliberately avoid the term *minimum detectable effect*: the classical MDE is a power-based quantity that depends on a significance level α and a power $1 - \beta$, whereas the half-width reported here is a precision measure conditional on the sample actually obtained. The two answer related but distinct questions.

The empirical curves are not constrained to be monotone, because each point is obtained from a finite bootstrap sample; small local reversals should be read as Monte Carlo variability rather than as evidence that additional independent versions systematically reduce precision. At 1000 replicates per point all four curves in Table 4 are monotone in the range reported; an earlier run with only 80 replicates per point produced a local reversal on the BugsInPy project-level curve, which disappeared once the replicate count was raised. This supports the Monte Carlo reading.

Reading the conservative (project-level) columns: resolving an effect of 0.05 requires on the order of 150 independent fault versions (149 on Defects4J

Table 4: Resolution of the estimate, expressed as the half-width of the 95% bootstrap interval, by number of independent fault versions. Required counts are read off a log–log line fitted to each curve over its stable range (at least 30 versions); values marked *Extrap.* lie beyond the resolution the curve actually measures, and are model-based extrapolations rather than empirically observed thresholds.

Independent	Defects4J		BugsInPy	
versions	Version-level	Project-level	Version-level	Project-level
20	1.437	1.461	0.520	1.173
30	0.468	0.459	0.250	0.224
40	0.235	0.320	0.183	0.158
50	0.136	0.197	0.139	0.114
70	0.083	0.138	0.091	0.080
100	0.045	0.085	—	—
Versions required for a target half-width				
Target = 0.10	65	90	65	58
Target = 0.05	96	149 *Extrap.*	117 *Extrap.*	101 *Extrap.*
Target = 0.02	160 *Extrap.*	293 *Extrap.*	251 *Extrap.*	213 *Extrap.*

and 101 on BugsInPy), and resolving 0.02 requires on the order of 250 (293 and 213 respectively). These are order-of-magnitude readings from fitted curves, not observed crossing points; at a target of 0.02 every entry is an extrapolation. Below roughly 30 versions the estimator is not merely imprecise but unstable, with half-widths between 0.2 and 1.5 and, in the smallest samples, values above 100—the fixed-effect model cannot estimate project effects reliably and the between-fold variance explodes.

These requirements are *conditional*: they follow from the project composition of these two benchmarks and from the resampling hierarchy chosen, and they are design-based guidance for sizing a similar study rather than a universal power requirement. A study with a different number of projects, or with project-level variation unlike that of Defects4J and BugsInPy, will land on different numbers; the curves in Table 4 are the transferable object, and the version counts we read off them are illustrations.

4.4. Interpretation

The practical consequence is not that behavioural features are worthless. It is that *this design cannot tell*: at conventional sample sizes, the difference

Table 5: Probability that an analysis reports a $\Delta R^2 > 0.02$ increment, when the feature's true within-project effect is exactly zero. Shown for the 400-instance cell; 2000 replications per cell. The grid varies projects and versions together, always with two subsets per version: 70 instances = 7 projects × 5 versions × 2, 200 = 10 × 10 × 2, 400 = 10 × 20 × 2. Monte Carlo standard error is at most 0.011 on any reported rate.

	Pooled analysis			FE
Confounding strength ρ	> 0.02	> 0.05	> 0.10	> 0.02
0.0	0.21	0.07	0.01	0.01
0.3	0.32	0.15	0.03	0.00
0.6	0.58	0.34	0.13	0.01
0.9	0.82	0.64	0.38	0.01

between a model with and without the candidate block is of the same order as the uncertainty of the difference itself, and on Defects4J the sign of the point estimate depends on an analyst choice (which cross-validation scheme) that is rarely reported. A study reporting a pooled ΔR^2 of +0.087 together with a nominally small p-value is not necessarily wrong; it is reporting a quantity whose interval contains both a meaningful improvement and a meaningful degradation, and whose value under within-project comparison reverses.

5. RQ2: Can project confounding alone produce such claims?

The empirical result above is consistent with confounding, but the observed data cannot by itself separate mechanism from noise. We therefore run a parameterised simulation whose data-generating process contains *no* within-project effect of the candidate feature, and whose other parameters are calibrated on the Defects4J data (between-project and within-project standard deviations, residual standard deviation).

The single free parameter is ρ, the correlation between a project's mean value of the candidate feature and its baseline fault-detection rate. $\rho = 0$ denotes no *linear* association between project-level feature means and baseline detection rates, and $\rho = 1$ denotes perfect positive linear alignment; neither endpoint is a statement about how well a classifier could recover projects from the feature. Crucially, ρ is directly estimable in any real study from project-level means, so this is a sensitivity analysis an author can run on their own data.

Table 5 and Figure 2(b) give the result. Two readings matter. At $\rho = 0$ the pooled analysis already reports an increment in 21% of runs: that is the estimator's own noise floor, and it alone means a single reported $\Delta R^2 > 0.02$ is weak evidence. As ρ grows, the rate rises to 82%. It does *not* fall with sample size: at $\rho = 0.9$ the pooled rate is 0.54, 0.76 and 0.82 across the three sample sizes in our grid (70, 200 and 400 instances), and at $\rho = 0.6$ it is 0.39, 0.52 and 0.58, while at 400 instances the fixed-effect rate is 0.01 or below throughout. This pattern is consistent with a systematic confounding mechanism rather than a reduction in sampling variance: adding observations within the same project structure does not remove it. Project fixed effects reduce the rate to at most 1% and drive it towards zero as versions accumulate.

Choice of threshold. The materiality threshold is a judgement, not a quantity the data can supply, so we report the probability of a material increment at three thresholds (Table 5) rather than one. The effect of the threshold is confined to the intercept: at $\rho = 0$ the pooled rate falls from 0.21 to 0.01 as the threshold tightens from 0.02 to 0.10, while at $\rho = 0.9$ it remains between 0.38 and 0.82. The qualitative conclusion—that the pooled analysis manufactures increments at rates that rise steeply with confounding—therefore does not depend on the threshold. We adopt 0.02 as a *prespecified practical threshold* for the simulation, reporting 0.05 and 0.10 as sensitivity thresholds; the qualitative conclusions are unchanged across them. The value is chosen to be comfortably below the pooled increment our own Defects4J sample produces (+0.087), so that a study of that character would clear the bar even if its entire apparent effect were confounding. The resolution target of $\Delta R^2 = 0.05$ used in Table 4 is likewise set by the magnitudes this literature reports, including our own pooled estimates; because the resolution curve is continuous, a reader who prefers a different target can read the required sample size directly off it.

We stress the scope of this diagnostic. A rate of 0.21 under $\rho = 0$ is not a classical Type I error rate: the criterion $\Delta R^2 > 0.02$ is a practical-significance threshold applied to a cross-validated difference, so the figure is the probability that a zero-effect predictor clears a materiality bar, not the probability of rejecting a null hypothesis at a nominal level. We therefore refer to it throughout as the probability of reporting a material increment, and avoid the term *error rate*.

Table 6: Project classification from behavioural features alone, under version-grouped 5-fold splitting, on an imbalanced class distribution. All figures are fold means. *Majority* is the accuracy of always predicting the largest project. The classifier is a random forest (300 trees, `random_state` = 0, otherwise SCIKIT-LEARN defaults); no hyperparameter tuning was performed.

Benchmark	$1/k$	Majority	Accuracy	Balanced acc.	Macro F_1
Defects4J	0.143	0.372	0.892	**0.712**	0.705
BugsInPy	0.143	0.527	0.837	**0.766**	0.725

6. RQ3: Do the findings generalise across languages?

We repeated the study on BugsInPy, seven Python projects (Table 1), using the same outcome construction and the same analysis code.

Project fingerprint. A random forest is trained to predict the source project of a subset. The unit of analysis is the subset; because several subsets share a fault version, we report both an instance-level 5-fold split and a split *grouped by fault version*, the latter answering the stronger question of whether the fingerprint generalises to versions the classifier has not seen. Instance-level accuracies for Defects4J and BugsInPy respectively are 0.895 and 0.824 from behavioural features alone; 0.835 and 0.890 from classical metrics alone; 0.935 and 0.943 from all features. The version-grouped accuracies are essentially unchanged (0.892, 0.838, 0.943 for Defects4J and 0.837, 0.918, 0.946 for BugsInPy), so the fingerprint is not an artefact of subsets from the same version appearing on both sides of the split.

The projects are highly unbalanced, so accuracy alone is not a fair summary: the majority class accounts for 0.372 of Defects4J instances and 0.527 of BugsInPy instances, both well above the $1/7 \approx 0.143$ uniform-guessing reference. Reporting only accuracy would leave open how much of the fingerprint is class imbalance, so Table 6 adds imbalance-robust measures. Under version-grouped splitting, behavioural features reach a balanced accuracy of 0.712 on Defects4J and 0.766 on BugsInPy—roughly five times the 1/7 reference—and a macro F_1 of 0.705 and 0.725. Neither is inflated by predicting the largest project, and both far exceed the majority-class baseline. Behavioural features do not merely differ across projects: the difference survives both a held-out-version split and a strict imbalance correction.

Neither feature family is "safer" than the other—the ranking between

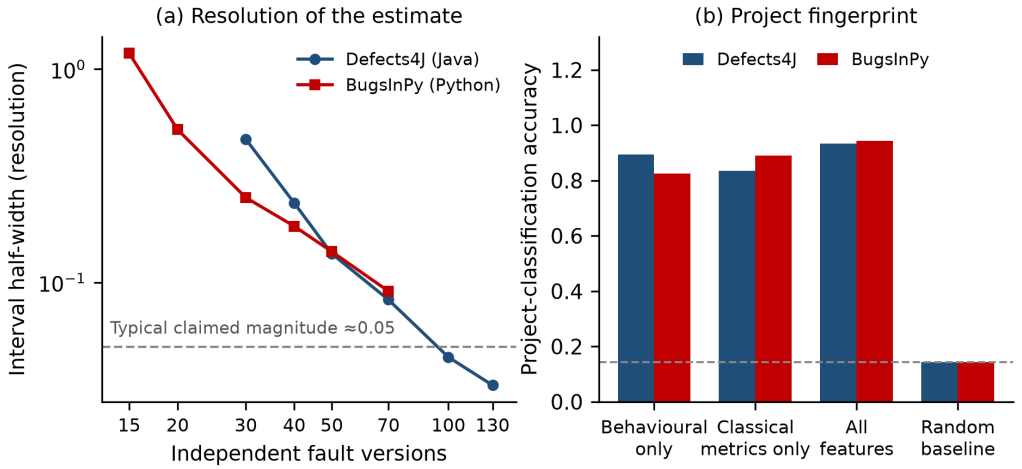


Figure 1: Cross-benchmark comparison. (a) Version-level resolution—the half-width of the 95% bootstrap interval—against the number of independent fault versions. The Defects4J curve reaches the $\Delta R^2 = 0.05$ reference at approximately 100 fault-version units (Table 4); the BugsInPy curve remains above it throughout its observed range. (b) Project-classification accuracy by feature set; both benchmarks exceed the uniform-random-guess baseline by a factor of about six.

them flips across benchmarks—which is itself an argument for controlling project identity rather than substituting one feature family for another.

Resolution. Table 2 and Table 3 show that the pooled analysis favours the larger model on both benchmarks, and that the correction for project identity shrinks the increment substantially on Defects4J ($+0.087 \rightarrow -0.011$) but only marginally on BugsInPy ($+0.018 \rightarrow +0.015$). The two benchmarks differ in whether the correction reverses the sign, and Section 7 shows that on BugsInPy the reversal appears once the dominant project is removed. On neither benchmark does the corrected increment exceed its own interval half-width.

7. RQ4: How sensitive are the conclusions to analytic choices?

A methodological claim is only useful if it survives the choices a reasonable analyst might make differently. We test four.

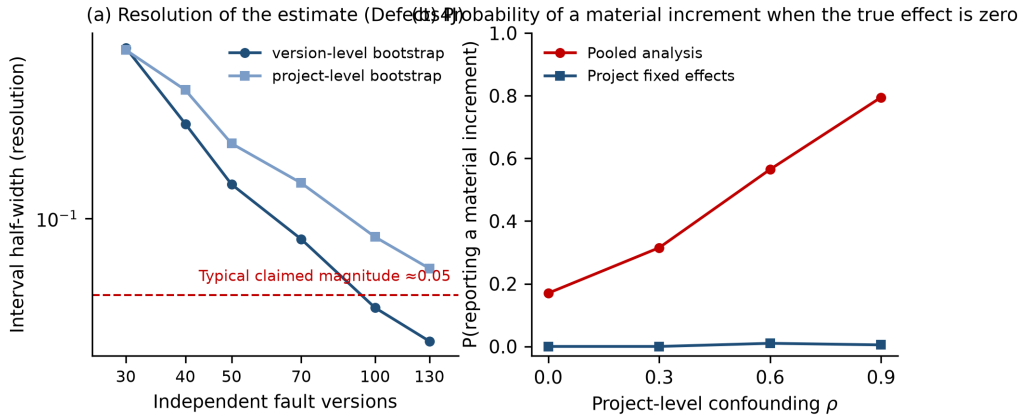


Figure 2: (a) Resolution curve for Defects4J. (b) Probability of reporting a material increment when the true effect is zero, as a function of project-level confounding ρ; project fixed effects suppress the artefact.

7.1. Leave-one-project-out

We recompute the model comparison on each leave-one-project-out sample (Table 7).

On Defects4J the within-project conclusion is stable: six of seven leave-one-out samples still rank the classical block first, and the increment stays within ± 0.06 in all seven. On BugsInPy the picture is the reverse: the full model ranks first in five of the seven leave-one-project-out samples, the remaining two going to the size-plus-behaviour block (`tqdm`) and to the classical block (`youtube-dl`); the BugsInPy result is therefore not an artefact of pooling alone. Its magnitude and sign nevertheless remain sensitive to the composition of projects (Section 7).

7.2. Removing the dominant project, and project-size thresholds

BugsInPy's version counts are highly unbalanced: `youtube-dl` contributes 43 of 82 versions. Removing it changes the result materially: the increment falls from $+0.015$ to -0.019 and the best model switches from full to classical (Table 8).

On BugsInPy the ≥ 2 versions row is exactly the removal of one project. `sanic` is the only project contributing a single fault version (three instances); every other project contributes at least two. Its removal is not cosmetic, because it is the one case in either benchmark where the fixed effect is not identified. `sanic` receives a project dummy that is estimable only in folds that

Table 7: Leave-one-project-out, with project fixed effects. "Best" is the model with the highest grouped-CV R^2 on the reduced sample.

Benchmark	Project removed	Best model	ΔR^2
Defects4J	Chart	Classical	−0.029
	Cli	Full	+0.002
	Codec	Classical	−0.003
	Compress	Classical	−0.010
	Csv	Classical	−0.000
	Lang	Classical	−0.057
	Math	Classical	−0.012
BugsInPy	black	Full	+0.037
	fastapi	Full	+0.017
	httpie	Full	+0.008
	sanic	Full	+0.025
	tornado	Full	+0.017
	tqdm	Size+behaviour	+0.020
	youtube-dl	Classical	−0.019

contain it; when its version is held out, that column is identically zero in the training folds, the minimum-norm least-squares solution sets the coefficient to zero, and its three held-out records are predicted at the reference project's level—3 of 245 held-out predictions (1.2 %). Excluding the project raises the increment from +0.015 to +0.025: neither the sign nor the selected model changes, and the effect of retaining it is to make the reported increment, if anything, more conservative.

Two honest readings follow. The Defects4J conclusion is robust to leave-one-project-out perturbations. The BugsInPy conclusion is not robust to dropping its largest one, and we report this rather than the version that survives: the cross-language agreement we can claim is that *the pooled analysis over-states the increment on both benchmarks*, not that the sign reversal reproduces on both.

Note that the two kinds of perturbation in Table 8 are not interchangeable. Raising a project-size threshold removes small projects while retaining the largest, and the BugsInPy increment consequently rises (from +0.015 to +0.045) as the sample concentrates on the projects whose pooled characteristics it was already driven by. Removing the dominant project does the

Table 8: Sensitivity to project composition. Cells give the best model under grouped CV with fixed effects, and the ΔR^2 of the full model over the classical block.

Benchmark	Sample	Projects	Best (FE)	ΔR^2
Defects4J	all	7	Classical	−0.011
	≥ 2 versions	7	Classical	−0.011
	≥ 5 versions	7	Classical	−0.011
	≥ 10 versions	4	Size+behaviour	+0.002
BugsInPy	all	7	Full	+0.015
	≥ 2 versions	6	Full	+0.025
	≥ 5 versions	4	Full	+0.045
	without `youtube-dl`	6	Classical	−0.019

Table 9: Scope of the outcome's denominator, grouped cross-validation on Defects4J. The two definitions differ only in whether the denominator is the project's whole fault set or the faults whose trigger class is present at the evaluated revision.

Denominator	Fixed effects	Size	Classical	Size+beh.	Full
Whole project	no	0.134	0.287	0.221	**0.391**
	yes	0.557	**0.592**	0.560	0.587
Revision-matched	no	0.211	0.586	0.240	**0.636**
	yes	0.543	**0.669**	0.560	0.668

opposite. The apparent conflict between the ≥ 5 and *without youtube-dl* rows is therefore not an inconsistency but the signature of the same underlying sensitivity: this benchmark's result is a function of which projects are in it.

7.3. Scope of the outcome's denominator

The outcome's denominator is the project's whole fault set, while each observation is drawn at a single fault version's fixed revision. On Defects4J only about half of a project's faults have their trigger class present in any one revision's test list (mean 32.7 of 65.8 faults per version). Restricting the denominator to the faults that are reachable at the evaluated revision raises the outcome's mean from 0.147 to 0.252; the two definitions correlate at $r = 0.83$ across instances.

The models' absolute R^2 values change considerably, but the comparison between them does not: the pooled analysis prefers the full model under

both definitions, project fixed effects reverse that preference under both, and the same model ranks first in all four cells as in Table 2. We retain the project-level denominator—it gives the outcome a target that does not depend on which fault the revision happens to fix—and report the alternative here rather than re-running the main analysis under it. This check is produced by `scripts/analyze_outcome_scope.py`.

7.4. Bootstrap hierarchy

The version-level and project-level bootstraps give interval widths of 0.056 and 0.120 on Defects4J, and 0.143 and 0.146 on BugsInPy (Table 3). The unit of resampling therefore changes reported uncertainty by a factor of up to two, and the two hierarchies do not even rank the benchmarks consistently. We report both throughout rather than choosing the more favourable one, and recommend that studies state their resampling unit explicitly.

7.5. Resolution as a design parameter

The version counts in Table 4 are presented as a function of the target effect rather than as a single number, because the answer depends on what the study wants to establish. The observed curves reach a half-width of approximately 0.10 at roughly 60–90 fault-version units, and a target of 0.02 lies beyond the sample sizes this literature typically reports.

8. A pre-submission protocol

The five checks below are cheap—none requires executing the test suites under study—and can be applied to any increment-in-R^2 claim.

1. **Report the nominal sample size and the independent-unit count.** Compute n_{nominal}, the number of subset instances, and $n_{\text{ind}} = |\{(\text{project}, \text{version})\}|$, the number of independent fault-version units. When the number of fault-version units is substantially smaller than the nominal instance count—as a practical heuristic, below half—either cluster the standard errors or move the analysis to the independent unit. We deliberately do not call n_{ind} an "effective sample size": the formal effective sample size of a clustered design also depends on the within-cluster dependence and on the cluster-size distribution, and we do not estimate it here. n_{ind} is a count of independent units, not a variance-adjusted sample size. Designing for more versions is more valuable than designing for more subsets per version.

2. **Screen predictors for project discriminability.** Fit a classifier predicting project from the predictor block alone. A block whose accuracy is a large multiple of the baseline ($1/k$ for k projects) will "predict" the outcome in any pooled analysis. Report the accuracy.
3. **Replicate under project fixed effects.** Demean within project (or add project dummies) and re-fit with standard errors clustered by version. Report pooled and within-project results side by side; if they disagree, the disagreement is itself a finding, and the within-project result is the one to build on.
4. **State the resampling unit and check the alternative.** Report whether the bootstrap resamples instances, fault versions, or projects; if the last is feasible, report it as well, since it reflects the more conservative dependence structure.
5. **Publish resolution, not just point estimates.** Report the interval half-width and state the smallest effect the design could have resolved. If the claimed effect is smaller than that, the design does not support the claim. Report the required number of independent fault versions for the target effect, as in Table 4.

Scope. Fixed effects remove any genuine project-level effect along with the confounding. Conclusions justified by this protocol are therefore about *within-project* validity. That is the correct scope for claims about a metric's diagnostic value for an individual project, but it must be stated: "this metric does not help within projects" and "this metric does not help" are different claims, and only the former follows from the analysis. Our conclusions concern incremental-validity analyses using trigger-test-based fault-detection proxies and do not establish metric validity under directly observed fault-detection outcomes.

9. Threats to validity

Construct validity. The outcome is computed from trigger-test metadata, which assumes that a suite containing a fault's trigger test detects that fault. This is the clean-program assumption [24]; it makes the outcome cheap and deterministic but abstracts away from fault masking and oracle quality, so our outcome is a proxy for detection rather than a direct measurement of it. The behavioural-matching predictors are syntactic proxies (assertion forms), not semantic measurements of whether a test asserts intended behaviour; our

conclusions are about the proxy, and they constitute evidence that syntactic proxies cannot carry the argument—not evidence against the semantic notion they were meant to index.

Internal validity. Subsets are constructed mechanically at fractions of the suite, so they vary in composition as well as in size. The behavioural block and size are correlated in the sample; we control for log size in all models, but residual dependence cannot be excluded. On BugsInPy, coverage is dominated by import-time execution in some projects (youtube-dl gains only 2 percentage points of coverage between a 10% and a 60% subset), which compresses within-project coverage variation and may understate coverage's within-project value. Finally, the two benchmarks differ in their classical metric block (BugsInPy has no mutation score), so the model comparison is not perfectly symmetric across languages.

External validity. Two benchmarks and fourteen projects is a small basis. Version counts are uneven within BugsInPy (43 down to 1), and one project contributes over half the Python instances; we report the leave-one-project-out and dominant-project-removed analyses (Section 7) precisely because of this. The extrapolated sample sizes in Table 4 rely on a log–log trend fitted to the observed curve and should be read as orders of magnitude. The version counts in Table 1 differ from the number of faults available in each project; coverage of the benchmark is therefore partial, and we do not claim it to be exhaustive.

Reliability of the instrument. Both pipelines are our own implementations. On Defects4J we do not use the framework's test-list facility (it does not exist); on BugsInPy we do not use the framework's per-fault checkout (one full clone per fault). Subset construction is therefore a measurement instrument of our own design, and its fidelity is supported by targeted checks rather than exhaustive validation. Sample sizes are reported after dropping records with incomplete metric measurement; the dropped records are itemised in the replication package.

Statistical conclusions. The bootstrap over seven projects is itself highly variable, and the project-level intervals in Table 3 should be read with that in mind. The simulation in Section 5 is a mechanism demonstration with calibrated dispersion, not an estimate of ρ in either benchmark; the observed project-level correlation in our Defects4J sample is weak ($r \approx +0.08$ across

seven projects, $p = 0.87$, itself imprecisely estimated), so the ρ sweep should be read as "if ρ were x" rather than as a description of these benchmarks.

10. Conclusion

A common design for validating test-adequacy metrics is incremental prediction. We have shown, on two independent benchmarks in two languages, that this instrument is not sharp enough for the conclusions typically drawn from it: the effects being claimed are of the same order as the uncertainty of their own estimates, their sign depends on unreported analyst choices, and project-level confounding alone can produce them in up to 82% of runs.

None of this implies that test-adequacy metrics are worthless, or that the existing studies are dishonest. It implies that the field's reporting convention—a point estimate and a p-value—does not carry the information readers need in order to judge the claim. The protocol in Section 8 is offered as a minimal remedy: five checks that cost no test executions, and a resolution statement that tells the reader what the study could have found.

CRediT authorship contribution statement

Junjie Chen: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data and code availability

All scripts, collected measurements, and analysis code are available at `https://github.com/JunjieChen0/test-adequacy-resolution`. Every number reported in this paper is produced by `scripts/analyze_all.py` from the shipped measurements; the repository contains both benchmarks' per-instance records, the analysis outputs, and a step-by-step reproduction guide.

References

[1] L. Inozemtseva, R. Holmes, Coverage is not strongly correlated with test suite effectiveness, in: Proceedings of the 36th International Conference on Software Engineering (ICSE), 2014, pp. 435–445. doi:10.1145/2568225.2568271.

[2] M. Papadakis, D. Shin, S. Yoo, D.-H. Bae, Are mutation scores correlated with real fault detection? a large scale empirical study on the relationship between mutants and real faults, in: Proceedings of the 40th International Conference on Software Engineering (ICSE), 2018, pp. 537–548. doi:10.1145/3180155.3180183.

[3] D. Schuler, A. Zeller, Assessing oracle quality with checked coverage, in: Proceedings of the 4th IEEE International Conference on Software Testing, Verification and Validation (ICST), 2011, pp. 90–99. doi:10.1109/ICST.2011.32.

[4] R. Niedermayr, E. Juergens, S. Wagner, Will my tests tell me if i break this code?, in: Proceedings of the International Workshop on Continuous Software Evolution and Delivery (CSED), 2016, pp. 23–29. doi:10.1145/2896941.2896944.

[5] O. L. Vera-Pérez, B. Danglot, M. Monperrus, B. Baudry, A comprehensive study of pseudo-tested methods, Empirical Software Engineering 24 (3) (2019) 1195–1225. doi:10.1007/s10664-018-9653-2.

[6] K. Jain, G. T. Kalburgi, C. Le Goues, A. Groce, Mind the gap: The difference between coverage and mutation score can guide testing efforts, in: Proceedings of the IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), 2023, pp. 102–113. doi:10.1109/ISSRE59848.2023.00036.

[7] S. Al Blwi, A. Ayad, B. Khaireddine, I. Marsit, A. Mili, Semantic coverage: measuring test suite effectiveness, in: Proceedings of the 18th International Conference on Software Technologies (ICSOFT), 2023, pp. 287–294. doi:10.5220/0012063900003538.

[8] A. Ayad, S. AlBlwi, A. Mili, Detecting faults vs. exposing failures: Orthogonal measures of test suite effectiveness, Journal of Systems and Software 230 (2025) 112450. doi:10.1016/j.jss.2025.112450.

[9] P. P. Paul, R. Holmes, Beyond coverage and kill scores: Empirically measuring test suite behavioural gaps, arXiv preprint arXiv:2606.10417 (2026).

[10] K.-Y. Liang, S. L. Zeger, Longitudinal data analysis using generalized linear models, Biometrika 73 (1) (1986) 13–22. doi:10.1093/biomet/73.1.13.

[11] C. Ni, X. Xia, D. Lo, X. Yang, A. E. Hassan, Just-in-time defect prediction on JavaScript projects: a replication study, ACM Transactions on Software Engineering and Methodology 31 (4) (2022) 1–38. doi:10.1145/3508479.

[12] E. H. Simpson, The interpretation of interaction in contingency tables, Journal of the Royal Statistical Society: Series B (Methodological) 13 (2) (1951) 238–241. doi:10.1111/j.2517-6161.1951.tb00088.x.

[13] J. M. Wooldridge, Econometric Analysis of Cross Section and Panel Data, 2nd Edition, MIT Press, Cambridge, MA, 2010.

[14] J. P. A. Ioannidis, Why most published research findings are false, PLoS Medicine 2 (8) (2005) e124. doi:10.1371/journal.pmed.0020124.

[15] J. Cohen, Statistical Power Analysis for the Behavioral Sciences, 2nd Edition, Lawrence Erlbaum Associates, Hillsdale, NJ, 1988.

[16] J. M. Hoenig, D. M. Heisey, The abuse of power: The pervasive fallacy of power calculations for data analysis, The American Statistician 55 (1) (2001) 19–24. doi:10.1198/000313001300339897.

[17] D. R. Roberts, V. Bahn, S. Ciuti, M. S. Boyce, J. Elith, G. Guillera-Arroita, S. Hauenstein, J. J. Lahoz-Monfort, B. Schröder, W. Thuiller,

D. I. Warton, B. A. Wintle, F. Hartig, C. F. Dormann, Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure, Ecography 40 (8) (2017) 913–929. doi:10.1111/ecog.02881.

[18] A. C. Cameron, J. B. Gelbach, D. L. Miller, Bootstrap-based improvements for inference with clustered errors, The Review of Economics and Statistics 90 (3) (2008) 414–427. doi:10.1162/rest.90.3.414.

[19] B. A. Kitchenham, E. Mendes, G. H. Travassos, Cross versus within-company cost estimation studies: A systematic review, IEEE Transactions on Software Engineering 33 (5) (2007) 316–329. doi:10.1109/TSE.2007.1001.

[20] T. Zimmermann, N. Nagappan, H. Gall, E. Giger, B. Murphy, Cross-project defect prediction: A large scale experiment on data vs. domain vs. process, in: Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE), 2009, pp. 91–100. doi:10.1145/1595696.1595713.

[21] N. Nagappan, T. Ball, A. Zeller, Mining metrics to predict component failures, in: Proceedings of the 28th International Conference on Software Engineering (ICSE), 2006, pp. 452–461. doi:10.1145/1134285.1134349.

[22] R. Just, D. Jalali, M. D. Ernst, Defects4j: A database of existing faults to enable controlled testing studies for java programs, in: Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA), 2014, pp. 437–440. doi:10.1145/2610384.2628055.

[23] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, E. L. Ouh, Bugsinpy: A database of existing bugs in python programs to enable controlled testing and debugging studies, in: Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2020, pp. 1556–1560. doi:10.1145/3368089.3417943.

[24] T. T. Chekam, M. Papadakis, Y. Le Traon, M. Harman, An empirical study on mutation, statement and branch coverage fault revelation that

avoids the unreliable clean program assumption, in: Proceedings of the 39th International Conference on Software Engineering (ICSE), 2017, pp. 597–608. doi:10.1109/ICSE.2017.61.